

Grammars

Programming Language Intro

Dr. Igor Steinmacher

A Hint of Syntax...

- The *syntax* of a programming language is a precise description of all its *grammatically* correct programs.

John is at home.

John are at home.

At home John is.

John is et home.

John home at is

A Hint of Syntax...

- The *syntax* of a programming language is a precise description of all its *grammatically* correct programs.

```
i = i + 2;
```

```
if (x>2) {  
    y = 3;  
}
```

```
b + 4 = 3
```

```
i = i 2 +;
```

```
if (x 2) {  
    y = 3;  
}
```

```
y = 3
```

Levels of Syntax

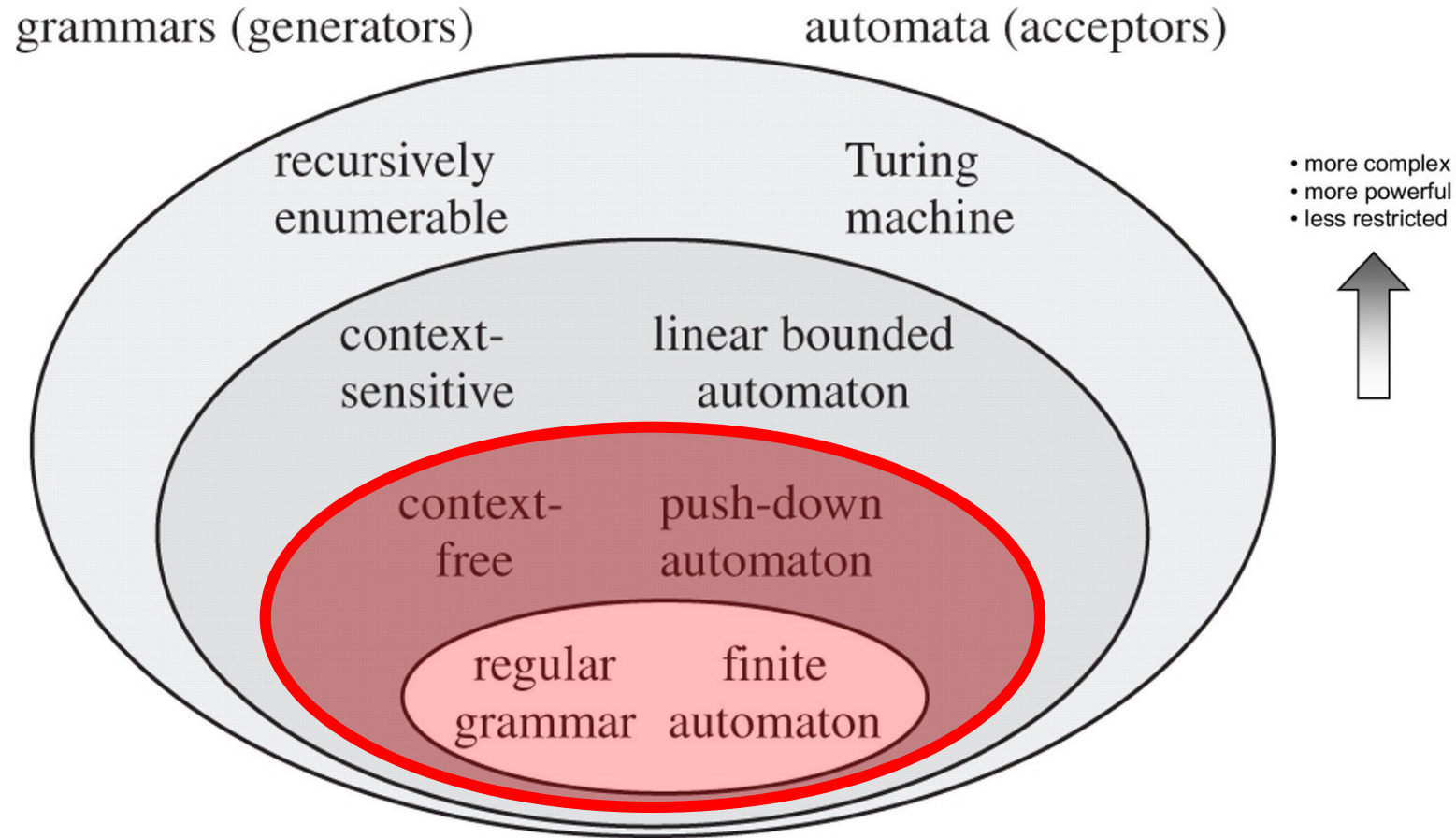
- **Lexical syntax**
 - all the basic symbols of the language (names, values, operators, etc.)
 - **Concrete syntax**
 - rules for writing expressions, statements, and programs.
 - **Abstract syntax**
 - internal representation of the program, favoring content over form.
-
- C: if (expr) ... discard ()
 - Ada: if (expr) then discard **then**

Grammar

- A *metalanguage* is a language used to define other languages.
- A *grammar* is a metalanguage used to define the syntax of a language.
- We will use grammars to define the syntax of a programming language.

Grammar

- *Noam Chomsky* hierarchy of classes of formal grammars



Backus-Naur Form Grammar

- John Backus and Peter Naur adapted Chomsky's Context-Free grammar to formally define ALGOL 60 language:
 - Backus-Naur Form or BNF Grammar
- Now used to define syntax of most major languages



BNF Grammar

- The metalanguage:
 - Terminal symbols: T
 - Non-terminal symbols: N
 - Start symbol: $S \in N$
- Set of Productions: P

$$A \rightarrow \omega$$

where $A \in N$ and $\omega \in (N \cup T)^*$

BNF: Example

- Consider the grammar:
 $binaryDigit \rightarrow 0$
 $binaryDigit \rightarrow 1$
- or equivalently:
 $binaryDigit \rightarrow 0 \mid 1$
- “|” is a metacharacter that separates alternatives (or)
- Single digit binary, all the productions go to a Terminal

BNF: Derivation

- Consider the grammar:

Integer $\rightarrow$ Digit | Integer Digit

Digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

- S = Integer
- We can *derive* any unsigned integer, like 742, from this grammar.

BNF: Derivation

- Deriving the number 742

1. Integer
2. Integer → Integer Digit
3. Integer → Integer 2
4. Integer → Integer Digit 2
5. Integer → Integer 4 2
6. Integer → Digit 4 2
7. Integer → 7 4 2

Integer → Digit | Integer Digit

Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Integer → Digit | Integer Digit

Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Integer → Digit | Integer Digit

Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Integer → Digit | Integer Digit

Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

BNF: Derivation (leftmost)

- Deriving the number 742

1. Integer

$\text{Integer} \rightarrow \text{Digit} \mid \text{Integer Digit}$

2. Integer $\rightarrow$ Integer Digit

$\text{Integer} \rightarrow \text{Digit} \mid \text{Integer Digit}$

3. Integer $\rightarrow$ Integer Digit Digit

$\text{Integer} \rightarrow \text{Digit} \mid \text{Integer Digit}$

4. Integer $\rightarrow$ Digit Digit Digit

$\text{Digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

5. Integer $\rightarrow$ 7 Digit Digit

$\text{Digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

6. Integer $\rightarrow$ 7 4 Digit

$\text{Digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

7. Integer $\rightarrow$ 7 4 2

$\text{Integer} \rightarrow \text{Digit} \mid \text{Integer Digit}$

$\text{Digit} \rightarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

Language

- Set of all strings that can be derived from the start symbol of a grammar (S)
- Example:
 - The grammar we used to exemplify can derive all non-negative integers

Integer $\rightarrow$ Digit | Integer Digit

Digit $\rightarrow$ 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Notations for Derivations

- $Integer \rightarrow^* 742$

Means that 742 can be derived in a finite number of steps using the grammar for Integer.

- $742 \in L(G)$

742 is a member of the language defined by grammar G.

- $L(G) = \{ \omega \in T^* \mid Integer \rightarrow^* \omega \}$

The language defined by grammar G is the set of all strings ω that can be derived from the start symbol Integer.

Increasing our Language

2561 → OK

1.67 → NO

0.32 → NO

- Real numbers are not part of our language!
- To improve, we need to do something that recognizes both integer and real, as numbers

Integer → Digit | Integer Digit

Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Increasing our Language

2561 → OK

1.67 → NO

0.32 → N

- Real numbers are not part of our language!
- To improve, we need to do something that recognizes both integer and real, as numbers

Number → Integer | Real

Real → Integer . Integer

Integer → Digit | Integer Digit

Digit → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

**See you in our next
Lesson!**